

Area & Power Optimized Hardware
Accelerator for Single Convolutional Kernel

Tel Aviv University

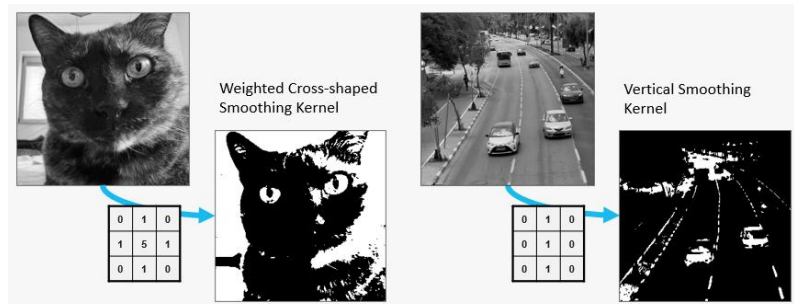
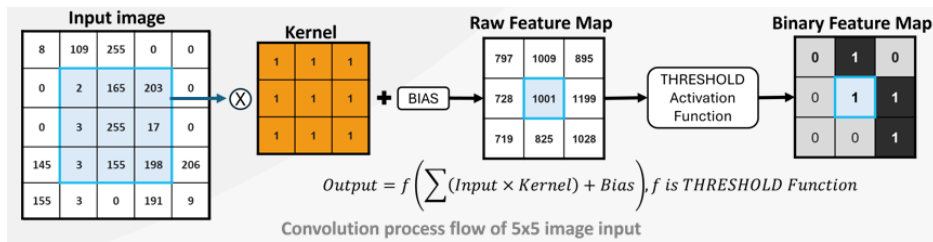
Electrical Engineering Gard Project

Overview

This project presents the design and implementation of a high-performance, area- and power-optimized hardware accelerator dedicated to performing real-time 2D spatial convolution using a single programmable 3x3 kernel. Designed specifically to target the TSMC 28nm technology node, the core architecture is highly optimized to strike a balance between minimal silicon footprint (Core Area) and ultra-low dynamic power consumption, making it ideal for energy-constrained Edge AI and vision processing systems.

To avoid the massive area and energy overhead associated with storing full video frames on-chip, the design implements a true Streaming Processing Paradigm. Pixels are ingested sequentially and processed on-the-fly. The memory architecture relies on a highly efficient 3x3 Matrix Physical Line Buffer Structure (configured as a cyclic, interleaved bank array). This unique layout provides parallel data access to nine distinct spatial positions simultaneously, enabling the extraction of a full 3x3 sliding window in a single clock cycle without structural hazards or memory port bottlenecks.

The mathematical datapath executes unsigned multiply-accumulate (MAC) operations across an adder tree, incorporates a programmable spatial bias, and passes the result through a non-linear activation stage. The final output is streaming and binarized into a 1-bit activation map by comparing the total sum against a programmable 20-bit threshold register.



System Requirements & Specifications

Interface Definition & Signal Description

System & Control Signals

- **clk (Input):** Master system clock driving all internal synchronous pipelines, multi-byte parameter registers, and the central finite state machine (FSM).
- **rst_n (Input):** Global asynchronous, active-low reset signal. For maximum silicon area savings, resets are strategically omitted from static parameters where state initialization is guaranteed via configuration loading.
- **cfg_wr_en (Input):** Synchronous Configuration Write Enable.
- **cfg_addr [3:0] (Input):** 4-bit configuration address bus used to target individual cell weights, the 8-bit network bias, or the individual bytes composing the multi-cycle activation threshold.
- **pixel_valid_in (Input):** Synchronous data valid flag from the host device.

Input Data Format & Parameters

- **input_bus [7:0] (Input):** A multiplexed 8-bit input data bus. During the initialization phase, it streams the 13-byte configuration payload. During the operational phase, it streams sequential 8-bit unsigned grayscale pixels arriving in continuous Raster Scan order.
- **Programmable Kernel Weights:** A spatial 3x3 filter matrix containing 9 weights total. Each weight coefficient is represented as an independent, fully programmable 8-bit unsigned integer value.
- **Programmable Bias:** An 8-bit unsigned offset appended directly into the MAC accumulation tree to scale the operating point of the spatial filter.
- **Programmable Threshold:** A 20-bit unsigned activation threshold. To minimize pin count, it is loaded into the device over 3 consecutive clock cycles via the 8-bit shared data bus using consecutive configuration addresses.

Output Data Interface

- **pixel_out (Output):** A 1-bit streaming binary classification/feature output. A value of 1'b1 denotes a positive feature activation, while 1'b0 denotes no activation.

- **pixel_valid_out (Output):** Synchronous control signal indicating the geometric and structural validity of the output pixel. It guarantees that only authentic processed data is marked as valid.

Performance & Microarchitectural Requirements

Throughput and Efficiency

- **Steady-State Throughput:** In steady-state execution mode, the architecture guarantees a throughput of 1 Output Pixel per Clock Cycle.
- **Zero-Stall Row Processing:** The interleaved line buffer configuration and dual-port register simulation ensure a completely non-blocking execution path. The pipeline processes active image rows continuously with zero structural stalls, maximizing hardware utilization.

Pipelining & Deterministic Latency

The module features completely deterministic execution latency dictated exclusively by structural frame filling and fixed arithmetic stages:

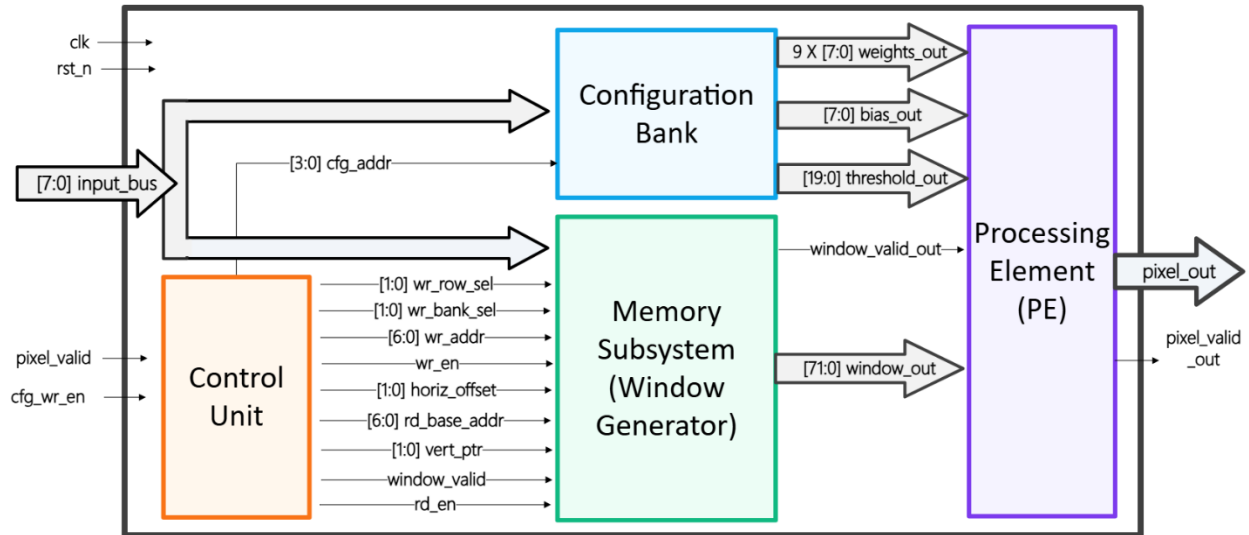
- **Line Buffer Warm-up Phase:** Processing begins with a fixed initial latency required to fill the first two rows of the cyclic line buffer and fetch the first two pixels of the third row ($2 \times \text{IMG_WIDTH} + 2$ valid clock cycles). This phase completely satisfies the initial geometric window requirement ($\text{row_cnt} \geq 2, \text{col_cnt} \geq 2$).
- **Processing Element Latency:** The physical multiply-accumulate and comparison datapath contains a structured 3-cycle pipeline latency from the moment a 3x3 window is extracted to the generation of the corresponding binarized output pixel.

Low-Power & Area Co-Optimization

- **Look-Ahead Integrated Clock Gating (ICG):** The FSM combinationally decodes a look-ahead `global_en` control path. This enables modern synthesis tools to automatically plant highly efficient clock-gating cells at the base of internal clock trees, shutting off toggling activity whenever the bus is idle.
- **Multi-Tiered Data Gating:** The 9 internal hardware multipliers and intermediate pipeline nodes are strictly data-gated; registers only update and toggle when `window_valid_in` is asserted, eliminating dynamic power waste during invalid frame padding intervals.

- **Footprint Reduction:** Area minimization is driven by the single, shared 8-bit data/configuration bus, exclusive reliance on space-saving unsigned logic libraries, and selective reset pruning across parameters.

Top-Level Block Diagram



The system comprises four main subsystems:

1. Control Unit (System Brain):

Manages the Finite State Machine (FSM), generates read/write memory addresses, and executes boundary masking logic (Valid Padding).

2. Memory Subsystem (Window Generator):

Responsible for buffering input data using a Cyclic Line Buffer and retrieving a parallel 3X3 window. Includes Re-ordering Logic for data alignment.

3. Processing Element (PE Core):

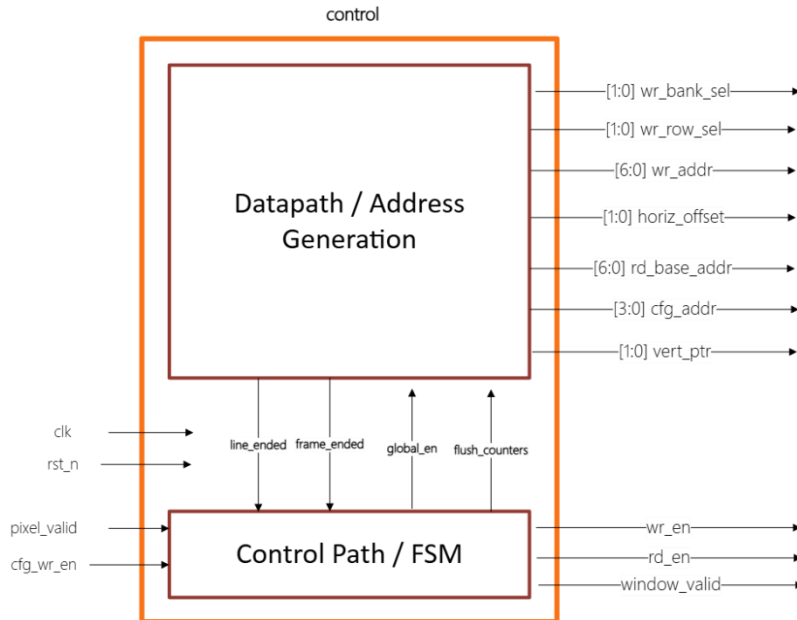
The arithmetic computational unit. Performs 9 multiply operations, summation via an Adder Tree, Bias addition, and Threshold comparison.

4. Configuration Bank:

Stores the programmable parameters (Weights, Bias, Threshold) received from the external interface prior to processing.

Micro Architecture

Control Unit (The System Brain)



General Description

The Control Unit (CU) acts as the central orchestrator of the chip. Its primary role is to manage system timing, generate memory read/write addresses, and synchronize the transition between the Configuration Phase and the Execution Phase. The module utilizes a set of internal counters and combinational logic to derive control signals for all other subsystems based on the incoming data stream.

Functionality

The block is responsible for the following core operations:

- **Configuration Management:** Autonomous generation of internal addresses for the Register Bank during parameter loading, utilizing the main column counter, eliminating the need for external address management.
- **Memory Address Space Management:** Translating the spatial pixel position (Row/Col) into physical addresses for the 9-Bank Interleaved Memory.
- **Window Alignment Control:** Generating selection signals for the Re-ordering Logic (MUXs) to ensure geometric consistency of the output window.

- **System State Management (FSM):** Monitoring the Warm-up phase and generating the final window_valid and rd_en signals for the processing core.

Interface Definition

Signal Name	Direction	Width	Description
clk	Input	1	System Clock.
rst_n	Input	1	Active Low Synchronous Reset. Resets all internal counters and FSM.
pixel_valid	Input	1	External indication that data on the Input Bus is valid for processing.
cfg_wr_en	Input	1	Configuration Write Enable. Increments the internal register address counter.
cfg_addr	Output	4	Target address for the Register Bank (0-12).
wr_addr	Output	7	Write Address within the physical bank (0-85).
rd_base_addr	Output	7	Base Read Address for window fetching.
wr_bank_sel	Output	2	Selects the specific Bank (0, 1, or 2) based on Col % 3.
horiz_offset	Output	2	Control signal for Horizontal Re-ordering MUXs.
wr_row_sel	Output	2	Selects the Write Row Group (A, B, or C).
vert_ptr	Output	2	Control signal for Vertical Re-ordering MUXs (Points to Top Row).
window_valid	Output	1	A valid window for the PE.
wr_en	Output	1	Write Enable for the Line Buffer (derived from global enable/state).
rd_en	Output	1	Read Enable for the Line Buffer (Active during WARM_UP and EXECUTE when pixel_valid is high).

Internal Logic & Sub-units

Internal Micro-Architecture The control logic is partitioned into four distinct sub-blocks, reflecting the cascaded counter topology:

A. Configuration Counter (Reused col_cnt)

- **Trigger:** Active when cfg_wr_en is high during the ST_CONFIG or ST_IDLE states.
- **Function:** Utilizes the lower bits of col_cnt to act as an incremental counter that generates the 4-bit address [3:0] cfg_addr.
- **Role:** Directs the loading of Weights (0-8), Bias (9), and Threshold components (10-12) into the Configuration Bank.

B. Column Counter (Col Counter 0-255)

- Trigger: Increments on every clock cycle where pixel_valid is high (during EXECUTE/WARMUP) or cfg_wr_en is high (during CONFIG).
- Function: Tracks the horizontal position of the incoming pixel within the frame or the configuration phase index.
- Outputs: Uses Modulo-3 and Division-by-3 logic to generate all horizontal control signals: wr_addr, rd_base_addr, wr_bank_sel, and horiz_offset.
- Cascading Signal: Generates a line_ended pulse when the counter wraps from 255 to 0. This pulse serves as the clock-enable for the Row Counter.

C. Row Counter (Row Counter 0-255)

- Trigger: Increments only when receiving the internal line_ended signal from the Column Counter AND pixel_valid is high. Crucially, to prevent system deadlocks, this increment tracks the valid flow of the 2D frame.
- Function: Tracks the vertical position of the current pixel.
- Outputs: Uses Modulo-3 logic to generate vertical control signals:
 - wr_row_sel: Determines which Cyclic Buffer (A, B, or C) is currently being written to.
 - vert_ptr: Indicates the "Top" row for the vertical re-ordering logic.

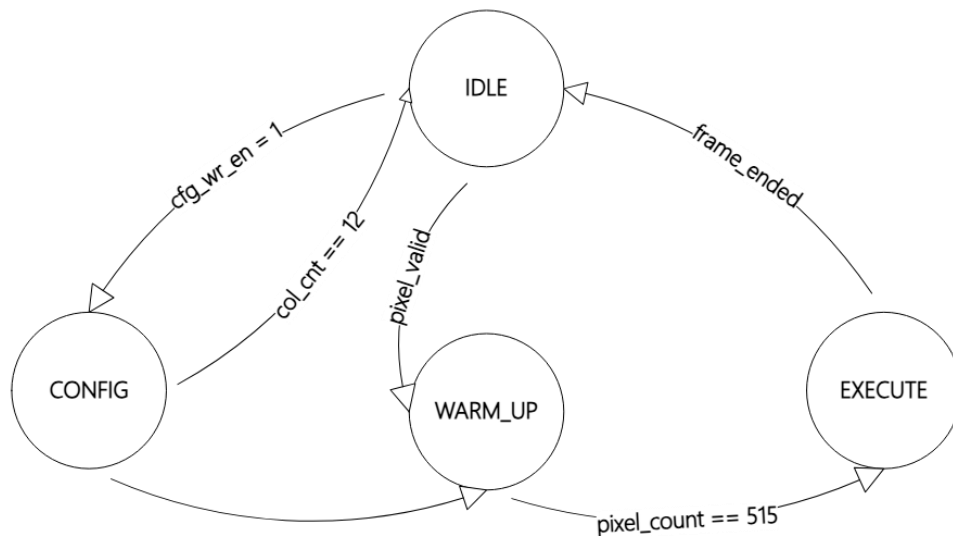
D. System FSM & Global Logic

- Function: Monitors the global state of the system.
- State Management: Controls transitions between IDLE, CONFIG, WARM_UP, and EXECUTE.
- **Boundary Logic (Window Fill Protection):** Determines the physical validity of the 3x3 convolution window. To prevent the calculation of "garbage windows", window_valid is strictly guarded by the physical fullness of the buffers:

$$\begin{aligned} window_valid = & (State == EXECUTE \text{ OR } State = \\ & = WARM_UP) \text{ AND } (pixel_valid == 1) \text{ AND } (row_cnt > \\ & = KERNEL_SIZE - 1) \text{ AND } (col_cnt > \\ & = KERNEL_SIZE - 1) \text{ Otherwise, } window_valid = 0. \end{aligned}$$

System FSM (Finite State Machine)

The Control Unit is driven by an FSM managing the frame processing lifecycle.



State Descriptions:

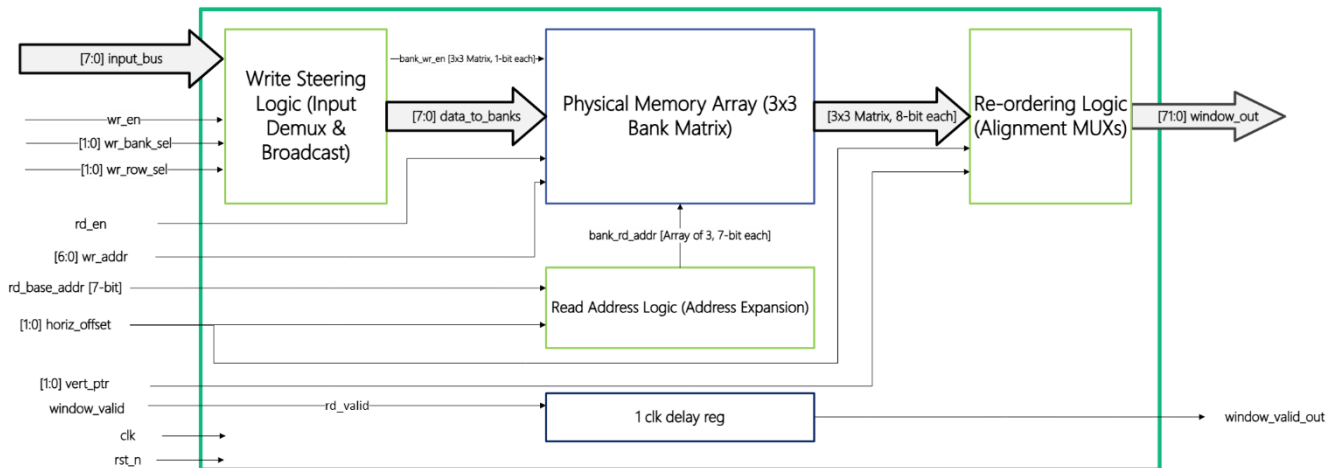
- **IDLE:** Default state after reset. Counters are zeroed or flushed upon entering. Waiting for `cfg_wr_en` (Config) or `pixel_valid` (Start Processing).
- **CONFIG:** Entered upon `cfg_wr_en`. The system increments `col_cnt` as an address counter to load parameters. Exits when it finishes writing the `TOTAL_CFG_CYCLES` (Weights + Bias + Threshold).
- **WARM_UP:** Entered upon first `pixel_valid`. The Line Buffer fills up (Rows A & B). `window_valid` remains '0' until the window boundaries are met.
- **EXECUTE:** Main processing state. `window_valid` is asserted to '1' (subject to boundary logic). The system supplies a 3X3 window every cycle. Returns to IDLE after a full frame (65,536 pixels) is received.

Current State	Condition	Next State	Output Actions
IDLE	cfg_wr_en == 1	CONFIG	Start incrementing col_cnt for config addressing.
IDLE	pixel_valid == 1	WARM_UP	Start incrementing row_cnt / col_cnt for spatial tracking.
CONFIG	col_cnt == (TOTAL_CFG_CYCLES - 1) AND cfg_wr_en == 1	IDLE	Lock registers, Ready for execution.
WARM_UP	row_cnt == (KERNEL_SIZE - 1) AND col_cnt == (KERNEL_SIZE - 1) AND pixel_valid == 1	EXECUTE	Enable processing mode (window_valid boundary met).
EXECUTE	frame_ended == 1 AND pixel_valid == 1	IDLE	De-assert window_valid, flush and reset counters.

Timing & Performance Summary

- **Zero Wait States:** The architecture enables a throughput of 1 Pixel/Clock with no stalls.
- **Single-Cycle Control:** Control signals, addresses, and selectors are generated in parallel with data arrival, ensuring continuous pipeline operation with no delays between loading and processing steps.
- **Robustness:** Built-in hardware protection against invalid data processing (Pipeline Fill Bug) during row transitions, filling phases, and host-induced pipeline stalls. Explicit flush-counter safety mechanisms handle back-to-back operations seamlessly.

Cyclic Line Buffer & Window Generator



General Description

The Cyclic Line Buffer & Window Generator acts as the high-bandwidth memory interface of the accelerator. Its primary function is to decouple the sequential input video stream from the parallel Processing Element (PE) core. Designed as a slave unit driven entirely by the Control Unit, the module buffers incoming pixels into an internal memory array and reconstructs a parallel, geometrically aligned 3x3 sliding window in every clock cycle. The architecture ensures Zero Stalls during active row execution and heavily prioritizes dynamic power reduction through granular clock gating.

Internal Micro-Architecture

The line buffer subsystem is partitioned into four highly optimized hardware sub-units, each handling a specific stage of the data routing and alignment process:

1. **Write Steering Logic (Input Demux & Broadcast):** A combinational routing network that broadcasts the incoming pixel data globally but uses a purely combinational 1-out-of-9 one-hot decoder to assert the write-enable (wr_en) for only the specifically targeted memory bank.
2. **Physical Memory Array (3x3 Bank Matrix):** The storage core, instantiated as 9 independent memory wrappers (3 Rows x 3 Banks). Crucially, this block integrates Latch-Based Integrated Clock Gating (ICG) at the individual bank level, ensuring that only banks actively participating in a read or write operation receive a toggling clock.
3. **Read Address Logic (Address Expansion):** A combinational address generation block that maps a single logical base address into 9 independent physical read addresses, dynamically handling cyclic buffer wrap-around scenarios without moving data physically.

4. **Re-ordering Logic (Alignment MUXs):** A zero-latency, two-stage multiplexing matrix that realigns the raw data fetched from the physical banks into a mathematically correct 3x3 geometric window for the processing core.

Memory Architecture & Mapping

To support a continuous throughput of 1 Window/Cycle without requiring highly multi-ported SRAMs, the memory utilizes a 9-Bank Interleaved Architecture.

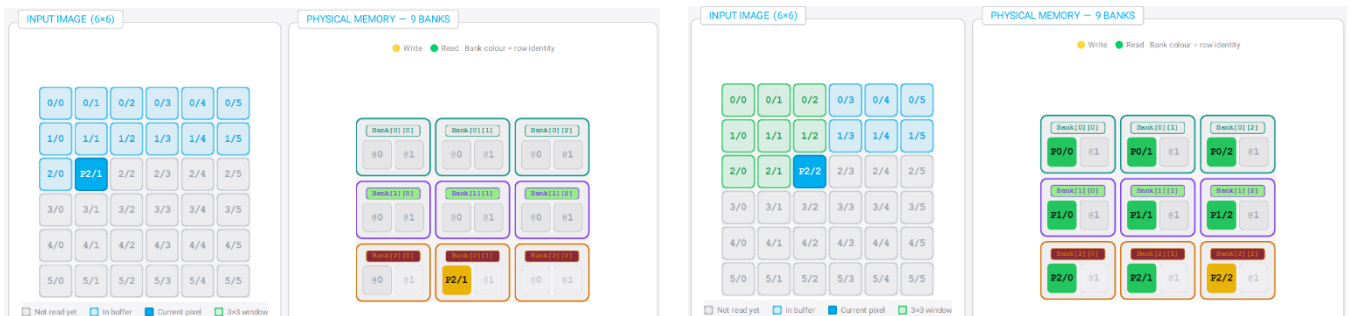
- **Logical Structure:** A 2D Cyclic Buffer storing 3 full image lines (Rows A, B, and C).
- **Physical Structure:** Pixels are distributed across the 9 banks based on their spatial location (Row Modulo 3 and Column Modulo 3).

Pixel Mapping Table

Each pixel $P(Row, Col)$ is stored in a specific bank according to the following mapping:

Row Buffer Group	Bank 0 (Cols: 0, 3, 6...)	Bank 1 (Cols: 1, 4, 7...)	Bank 2 (Cols: 2, 5, 8...)
Row Buffer A (Rows: 0, 3, 6...)	$P(0,0), P(0,3), P(0,6)...$ $P(3,0), P(3,3)...$	$P(0,1), P(0,4), P(0,7)...$ $P(3,1), P(3,4)...$	$P(0,2), P(0,5), P(0,8)...$ $P(3,2), P(3,5)...$
Row Buffer B (Rows: 1, 4, 7...)	$P(1,0), P(1,3), P(1,6)...$ $P(4,0), P(4,3)...$	$P(1,1), P(1,4), P(1,7)...$ $P(4,1), P(4,4)...$	$P(1,2), P(1,5), P(1,8)...$ $P(4,2), P(4,5)...$
Row Buffer C (Rows: 2, 5, 8...)	$P(2,0), P(2,3), P(2,6)...$ $P(5,0), P(5,3)...$	$P(2,1), P(2,4), P(2,7)...$ $P(5,1), P(5,4)...$	$P(2,2), P(2,5), P(2,8)...$ $P(5,2), P(5,5)...$

- **Architectural Advantage:** Any valid 3x3 sliding window on the image will always consist of 9 pixels that reside in 9 *distinct* physical banks. This guarantees immediate, collision-free retrieval in a single cycle.



Interface Definition

The table below summarizes the core interface signals of the Line Buffer subsystem (matching the `line_buffer_top` RTL module). Crucially, it details the mathematical formulas and generation logic managed entirely by the external Control Unit.

Signal Name	Direction	W	Description & Generation Logic
<code>clk</code>	Input	1	System Clock: Drives all synchronous reads, writes, and pipeline registers.
<code>rst_n</code>	Input	1	Active Low Reset: Clears internal pipeline alignment registers. Memory array contents are un-reset to save area.
<code>pixel_in</code>	Input	8	Incoming Pixel Data: Broadcasted globally to all 9 physical memory banks.
<code>wr_en</code>	Input	1	Global Write Enable: Activated by the Control Unit when <code>pixel_valid</code> is high.
<code>wr_row_sel</code>	Input	2	Write Row Select [0-2]: Determines which of the 3 cyclic row buffers (A, B, or C) receives the incoming pixel. Logic: <code>wr_row_sel = row_cnt % 3</code>
<code>wr_bank_sel</code>	Input	2	Write Bank Select [0-2]: Determines the specific physical bank within the selected row. Logic: <code>wr_bank_sel = col_cnt % 3</code>
<code>wr_addr</code>	Input	7	Physical Write Address [0-85]: The internal SRAM depth index where the pixel is stored. Logic: <code>wr_addr = col_cnt / 3</code>
<code>rd_base_addr</code>	Input	7	Read Base Address: The reference address used to fetch the current 3-pixel horizontal slice. Logic: It is a pipelined, delayed version of <code>wr_addr</code> to ensure correct alignment with the sliding window.
<code>horiz_offset</code>	Input	2	Horizontal Alignment Offset [0-2]: Indicates the physical bank that holds the "Left" geometric column of the current window. Used by the Address Expansion logic and Horizontal MUXs. Logic: Directly derived from a delayed <code>wr_bank_sel</code> .
<code>vert_ptr</code>	Input	2	Vertical Alignment Pointer [0-2]: Indicates which cyclic row (A, B, or C) represents the geometric "Top" row of the current window. Used by the Vertical MUXs.

			Logic: <code>vert_ptr = (wr_row_sel == 2) ? 0 : wr_row_sel + 1</code>
rd_valid	Input	1	Incoming Valid Flag: Driven by the boundary-protection logic in the Control Unit. Asserts only when the buffers are sufficiently full to prevent garbage calculations.
window_out	Output	72	Aligned Output Window: A fully aligned vector containing 9 pixels (9x8-bit). Generated combinatorially by the Re-ordering logic.
window_valid_out	Output	1	Window Valid Output: A one-cycle delayed pass-through of rd_valid. Acts as the valid_in trigger for the Processing Element (PE).

Detailed Functional Description

A. Write Steering Logic (Input Demux)

Instead of using deep shift-registers (which consume massive power due to continuous data toggling), this module uses a Broadcast & Steer strategy.

- **Data Path:** The pixel_in is broadcasted combinatorially to the inputs of all 9 banks.
- **Control Path:** Based on wr_row_sel and wr_bank_sel, a dedicated decoder asserts the bank_wr_en for only one bank per cycle. The RTL includes formal SystemVerilog Assertions (SVA) to mathematically prove that multiple banks cannot be written to simultaneously.

B. Physical Memory Array (3x3 Bank Matrix)

The storage core is fundamentally constructed from 9 independent Single-Port SRAM blocks, arranged in a 3x3 matrix.

- **Function & Bandwidth:** Because each bank has its own dedicated address, data, and enable lines, the architecture achieves the massive bandwidth of a multi-ported memory while using standard, high-density single-port SRAMs. The system can safely perform one targeted Write operation (to a single bank) and nine simultaneous Read operations (from all banks) in the exact same clock cycle.
- **Power Optimization:** To aggressively reduce dynamic power, each individual bank is guarded by a dedicated Latch-Based Integrated Clock Gating (ICG) cell. A bank's local clock will only toggle if it is actively selected for a write or participating in a global read.

- **RAW Hazard Protection:** The memory wrappers include a "Write-First Bypass" mechanism. If a read and write target the exact same address simultaneously (a Read-After-Write collision), the incoming pixel is forwarded directly to the output, ensuring data coherency with zero pipeline stalls.

C. Read Address Logic (Boundary Expansion)

To extract a 3×3 window, the system must fetch a 3-pixel horizontal slice from each row buffer. However, because the buffers are cyclic, a logical 3-pixel window will often cross the physical boundary of the memory array (wrap-around).

This combinational logic block solves the wrap-around problem with zero latency by dynamically generating specific read addresses for each physical bank.

- **Input:** Receives a single `rd_base_addr` (calculated as `col_cnt / 3`) from the Control Unit.
- **Logic ("Look-Ahead" Mechanism):** It expands the base address into 3 independent addresses based on the window's starting column (`horiz_offset`):
 - **If `horiz_offset == 0` (Aligned):** The physical window matches the logical window. All banks fetch from `rd_base_addr`.
 - **If `horiz_offset == 1` (Offset by 1):** The window starts at Bank 1. Bank 0 must wrap around to fetch the next logical pixel, reading from `rd_base_addr + 1`. Banks 1 and 2 read normally from `rd_base_addr`.
 - **If `horiz_offset == 2` (Offset by 2):** The window starts at Bank 2. Banks 0 and 1 must wrap around, reading from `rd_base_addr + 1`. Bank 2 reads normally from `rd_base_addr`.

D. Re-ordering Logic (Alignment MUXs)

Data fetched directly from the 9 physical banks corresponds to their hardwired physical locations, which becomes geometrically scrambled due to the cyclic nature of the buffer. This combinational module acts as a zero-latency crossbar, realigning the raw data into a perfectly structured 3×3 spatial window using a two-stage MUX network:

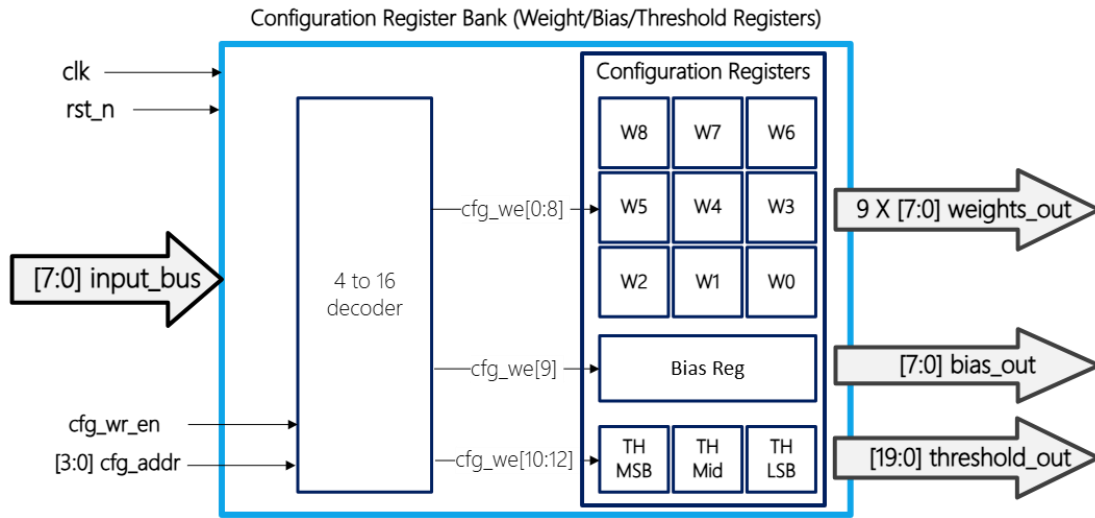
1. **Stage 1 - Horizontal Realignment (Column Rotation):** Uses the `horiz_offset` signal to dynamically shift the columns within each row. This guarantees that the logical "Left" column of the current window is always routed to the correct Index [0] (MSB) position of the output matrix, regardless of which physical bank it was read from.

2. **Stage 2 - Vertical Realignment (Row Rotation):** Because the 3 row buffers (A, B, C) are written cyclically, the logical "Top" row of the image changes continuously. This stage uses the `vert_ptr` signal to rotate the three fully-formed rows, ensuring that the buffer currently holding the top of the geometric window is physically routed to the Top Row (`win_matrix[0]`) of the final `window_out` vector.

Timing & Performance

- **Latency:** The line buffer has an internal read latency of 1 clock cycle (synchronous read). Pipeline registers are utilized on the control signals (`horiz_offset_d1`, `vert_ptr_d1`) to maintain perfect alignment with the data path.
- **Throughput:** 1 Window per Clock Cycle (during the Execution phase).
- **Stalls:** Continuous operation with **Zero Stalls**, enabled by the Write-First bypass and interleaved addressing scheme.

Configuration Register Bank



Functional Description

The Configuration Register Bank serves as a programmable, synchronous parameter storage unit (operating via a Slave Interface) that establishes the operational network state of the accelerator core. Its primary structural role is to bridge the bandwidth and performance gap between the narrow, resource-sharing external host bus (**8-bit**) and the deep mathematical pipeline inside the Processing Element (PE). The PE demands instantaneous, non-blocking, parallel access to all spatial filter parameters simultaneously to preserve a continuous steady-state throughput of 1 Pixel/Clock Cycle.

The module encapsulates and isolates the following fully programmable parameters:

- **Kernel Weights:** Nine independent 8-bit unsigned values representing a 3x3 spatial convolution matrix (72 bits total).
- **Spatial Bias:** A single 8-bit unsigned offset appended directly to the PE's accumulation tree to alter the activation function's operational threshold point.
- **Activation Threshold:** A 20-bit unsigned decision parameter used to binarize the accumulated matrix sum into a 1-bit classification map. It is parsed and streamed sequentially over 3 consecutive configuration clock cycles.

Interface Definition

Signal Name	Direction	Bit-Width	Data Type	Description
clk	Input	1	logic	Master System Clock.
rst_n	Input	1	logic	Global asynchronous, active-low reset signal.
cfg_wr_en	Input	1	logic	Configuration Write Enable control signal generated by the central FSM.
cfg_addr	Input	4	logic [3:0]	Configuration Address pointer targeting destination registers (Range: 4'h0 to 4'hC).
input_bus	Input	8	config_t	Shared 8-bit input bus streaming the configuration bytes during ST_CONFIG.
weights_out	Output	72	kernel_weights_t	Packed vector containing all 9 spatial weights, routed in a parallel, static grid directly into the PE multipliers.
bias_out	Output	8	weight_t	Registered bias integer value driven continuously into the adder accumulation stage.
threshold_out	Output	20	accum_t	Truncated 20-bit activation threshold value fed directly to the activation comparator block.

Advanced Spatial Weight Remapping (180° Geometric Matrix Flip)

Mathematically, a valid 2D spatial convolution operation ($Y = X * W$) requires the filter kernel (W) to be inverted both horizontally and vertically (180° flip) prior to element-wise multiplication. While software frameworks often bypass this by performing cross-correlation, an ASIC implementing true convolution must execute this geometric flip.

Implementing this flip dynamically inside the Processing Element (PE) using multiplexers or routing matrices introduces severe area, power, and timing overheads into the high-frequency datapath. To eliminate these penalties, the architecture resolves the inversion entirely during the static configuration phase via a zero-cost look-up indexing strategy:

$$Internal\ Index = 8 - cfg_addr$$

As the host streams the kernel weights linearly, the write decoder automatically assigns them to inverted register locations (e.g., W_0 is stored in `weights_out[8]`). During active execution, the PE simply performs direct, parallel element-wise multiplication (`pixels[i] * weights_out[i]`). This static remapping shifts structural complexity out of the continuous execution loop, resulting in a Zero-Gate, Area-Optimized datapath.

Theory of Operation

The module operates in two distinct phases:

Phase 1: Configuration Mode (Sequential Loading)

In this phase, the system receives parameters (such as weights and threshold) and stores them. When the write enable signal (`cfg_wr_en`) is asserted, the registers sample the incoming data (8 bits at a time) on every rising clock edge and map it to the correct internal register based on the address (`cfg_addr`).

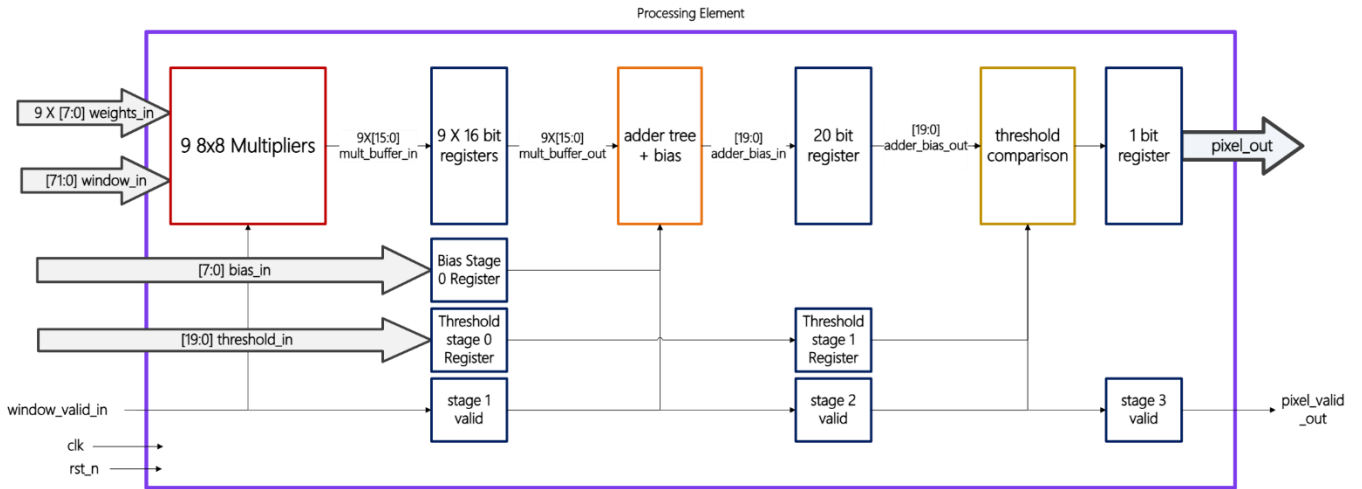
- **Loading Weights & Bias (Addresses 0–9):** The 9 weights and 1 bias (each 8 bits wide) are stored directly in their designated registers. Importantly, the mathematical inversion of the weights (remapping) is performed during this step.
- **Threshold Assembly (Addresses 10–12):** Because the threshold value is 20 bits wide and the input bus is only 8 bits, it must be assembled over 3 clock cycles. The system captures the Least Significant Byte (LSB), the Middle Byte, and the Most Significant Byte (MSB), concatenating them into a temporary 24-bit register (`thresh_reg[23:0]`).
- **Total Time:** The entire configuration process takes exactly 13 clock cycles.

Phase 2: Execution Mode (Parallel Static Distribution)

Once parameter loading is complete, the register bank transitions to a "passive" mode (acting like a static look-up table). The `cfg_wr_en` signal drops to 0, which freezes the clock to these registers (via Integrated Clock Gating) to save energy, effectively reducing dynamic switching power to zero. From this point forward, all stored data (`weights_out`, `bias_out`, and `threshold_out`) is continuously and simultaneously driven to the Processing Element (PE). This enables immediate, non-blocking execution (Zero-Latency), which is essential for maintaining the 1 pixel per clock cycle throughput.

Address (<code>cfg_addr</code>)	Target Parameter	Internal Slice	Re-mapped Destination
0-8	Weights $W_0 - W_8$	[7:0]	<code>weights_out[8]</code> down to [0]
9	Bias	[7:0]	<code>bias_out[7:0]</code>
10	Threshold Byte 0 (LSB)	[7:0]	<code>thresh_reg[7:0]</code>
11	Threshold Byte 1 (Mid)	[7:0]	<code>thresh_reg[15:8]</code>
12	Threshold Byte 2 (MSB)	[3:0]	<code>thresh_reg[19:16]</code>
13-15	Reserved / Unused	–	–

Processing Element (PE)



General Description

The Processing Element (PE) serves as the central computational unit of the accelerator. It is responsible for performing the Multiply-Accumulate (MAC) operation on the 3X3 sliding window.

The block executes 9 parallel multiplications (Pixel × Weight), sums the partial products, adds a programmable Bias, and finally compares the result against a Threshold to determine the binary output value (0 or 1). Additionally, it outputs the intermediate MAC sum for debugging or pooling purposes.

Mathematical Operation:

$$Sum = Bias + \sum_{i=0}^8 Pixel_i \times Weight_i$$

$$Pixel\ Out = \begin{cases} 1, & \text{if } Sum > Threshold \\ 0, & \text{otherwise} \end{cases}$$

Interface Definition

Signal Name	Direction	Width	Description
clk	Input	1	System Clock.
rst_n	Input	1	Active Low Synchronous Reset.
window_valid_in	Input	1	Input validity flag (from Control Unit / Window Generator).
pixels_	Input	72	Input Window: Flattened vector containing 9 pixels (9X8-bit).
weights	Input	72	Kernel Weights: Flattened vector containing 9 weights (9X8-bit).
bias_in	Input	8	Bias value from Register Bank.
threshold	Input	20	Activation Threshold value.
window_valid	Input	1	Input validity flag (from Window Generator).
pixel_out	Output	1	Classification Result: Final binary output (0 or 1).
pixel_valid_out	Output	1	Output Valid: Indication that the calculation is complete and pixel_out contains valid data.

Functional Description

The computation logic is implemented using a 3-Stage Pipelined Architecture to maintain high operating frequency and throughput. As shown in the RTL, pipeline registers are inserted between computational stages to balance the logic delay:

Stage 1: Multiplication

The block instantiates 9 parallel multipliers.

- **Power Optimization (Data Gating):** Input pixels and multipliers are strictly gated with the window_valid_in signal. If the window is invalid (e.g., during row transitions or pipeline filling), the multipliers' input registers are not toggled, preventing unnecessary combinational switching and saving significant dynamic power.
- **Operation:** Each multiplier accepts one pixel and one static weight.
- **Pipeline Registers:** The 9 product results are latched into the first pipeline register array (mult_q). Simultaneously, the valid signal (s1_valid), the bias, and the threshold

are also pipelined into s1_bias and s1_threshold to keep them aligned with the data phase.

Stage 2: Balanced Adder Tree & Bias

- **Summation & Bias:** The combinational logic sums up all 9 partial products along with the pipelined bias value (s1_bias) into a single result (acc_q_res).
- **Width:** The internal bus scales to the accumulator width (accum_t) to prevent overflow.
- **Pipeline Registers:** The total sum is latched into the second pipeline register (acc_q). The valid flag (s2_valid) and the threshold (s2_threshold) are propagated accordingly.

Stage 3: Thresholding (Activation)

The accumulated sum is compared against the threshold.

- **Logic:** if (acc_q > s2_threshold) then the activation bit is 1, else 0.
- **Output Registers:** The final 1-bit binary result is stored in the output flip-flop (act_q). Alongside it, the raw sum is also registered into sum_q. The valid signal completes its shift into s3_valid. These registers drive the final output ports (pixel_out, sum, and pixel_valid_out).

Control Flow (Valid Synchronization)

To ensure data integrity within the pipeline:

- The window_valid_in signal propagates through a 3-stage shift register (s1_valid → s2_valid → s3_valid), perfectly matching the arithmetic pipeline depth.
- The resulting pixel_valid_out is asserted in exact synchronization with the corresponding valid results appearing at pixel_out and sum.

Handling Row Transitions (The "Bubbles")

The system natively handles the transition between image rows without stalling the clock:

- **Input Stream:** The input continues uninterrupted. However, at the start of a new row, the internal Window Valid signal drops to '0' because a complete 3x3 neighborhood cannot be formed until the padding/boundary conditions are met.

- **Output Stream:** This "Invalid Gap" (bubble) propagates cleanly through the valid shift-registers and manifests at the output exactly 3 cycles later.
- **Recovery:** The system automatically recovers and resumes valid output generation once the valid shift-conditions are met, ensuring a continuous throughput of 1 pixel/cycle for the valid body of the image.

Timing Diagrams

Signal / Cycle	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
System State	CONFIG	CONFIG	CONFIG	CONFIG	CONFIG	CONFIG	CONFIG	CONFIG	CONFIG	CONFIG	CONFIG	CONFIG	CONFIG	WARM_UP	WARM_UP	WARM_UP
Input Bus	Weight 0	Weight 1	Weight 2	Weight 3	Weight 4	Weight 5	Weight 6	Weight 7	Weight 8	Bias	Th_LSB	Th_MID	Th_MSB	Pixel 0	Pixel 1	Pixel 2
Window Valid	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
PE Stage 1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
PE Stage 2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
PE Stage 3	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
Output Data	Low	Low	Low	Low	Low	Low	Low	Low	Low	Low	Low	Low	Low	Low	Low	Low
Pixel Valid Out	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Signal / Cycle	520	521	522	523	524	525	526	527	528	529	530	531	532	533	534	535
System State	WARM_UP	WARM_UP	WARM_UP	WARM_UP	WARM_UP	WARM_UP	WARM_UP	WARM_UP	EXECUTE	EXECUTE	EXECUTE	EXECUTE	EXECUTE	EXECUTE	EXECUTE	EXECUTE
Input Bus	Pixel 507	Pixel 508	Pixel 509	Pixel 510	Pixel 511	Pixel 512	Pixel 513	Pixel 514	Pixel 515	Pixel 516	Pixel 517	Pixel 518	Pixel 519	Pixel 520	Pixel 521	Pixel 522
Window Valid	0	0	0	0	0	0	0	1 (Win 0)	1	1	1	1	1	1	1	1
PE Stage 1	-	-	-	-	-	-	-	-	Win 0	Win 1	Win 2	Win 3	Win 4	Win 5	Win 6	Win 7
PE Stage 2	-	-	-	-	-	-	-	-	-	Win 0	Win 1	Win 2	Win 3	Win 4	Win 5	Win 6
PE Stage 3	-	-	-	-	-	-	-	-	-	-	Win 0	Win 1	Win 2	Win 3	Win 4	Win 5
Output Data	Low	Low	Low	Low	Low	Low	Low	Low	Low	Low	Low	Res 0	Res 1	Res 2	Res 3	Res 4
Pixel Valid Out	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1

Signal / Col Index	254	255 (End)	0 (Next)	1 (Next)	2 (Start)	3	4	5	6	7
System State	EXECUTE	EXECUTE	EXECUTE	EXECUTE	EXECUTE	EXECUTE	EXECUTE	EXECUTE	EXECUTE	EXECUTE
Row Position	Body	Right Edge	Left Edge	Left Edge	Valid Start	Body	Body	Body	Body	Body
Input Bus	P(254)	P(255)	P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)
Window Valid	1	1	0	0	1	1	1	1	1	1
PE Stage 1	Active	Active	Bubble	Bubble	Win(2)	Win(3)	Win(4)	Win(5)	Win(6)	Win(7)
PE Stage 2	Active	Active	Active	Bubble	Bubble	Win(2)	Win(3)	Win(4)	Win(5)	Win(6)
PE Stage 3	Active	Active	Active	Active	Bubble	Bubble	Win(2)	Win(3)	Win(4)	Win(5)
Output Data	Res(251)	Res(252)	Res(253)	Res(254)	Res(255)	Invalid	Invalid	Res(2)	Res(3)	Res(4)
Pixel Valid Out	1	1	1	1	1	0	0	1	1	1